

第三章总结

Chapter 3 的 单纯形法

总目标：把“顶点最优”变成“可执行算法”

第二章已经给出一个关键理论：

如果标准形线性规划有最优解，那么至少存在一个 **基本可行解** (basic feasible solution, BFS) 是最优的。

第三章做的事，就是把这句话真正“落地”为算法。

也就是说，它要回答 6 个问题：

1. 当前 BFS 怎么判断是不是最优？
2. 如果不是最优，怎么找到更好的相邻 BFS？
3. 这些步骤如何高效实现？
4. 如果出现退化，为什么会麻烦，怎么防止循环？
5. 如果一开始根本不知道初始 BFS，怎么办？
6. 这个算法几何上在干什么，理论上又快不快？

所以你可以把这一章看成一句话：

单纯形法 = 在可行域的顶点之间，有规则地换基 (pivot)，直到满足最优性条件。

一、本章的统一问题形式

本章研究的标准形线性规划是

$$\min c^T x$$

subject to

$$Ax = b, \quad x \geq 0$$

其中 A 是一个 $m \times n$ 矩阵，并且满足 $\text{rank}(A) = m$ 。

可行域记作

$$P = \{x \in \mathbb{R}^n \mid Ax = b, x \geq 0\}.$$

在这个框架下，最核心的对象有 5 个：

1. 基 (basis)

从 A 的列中选出 m 个线性无关的列，组成基矩阵 B 。

2. 基本解 (basic solution)

令非基变量为 0，再由

$$x_B = B^{-1}b$$

确定基变量。

3. 基本可行解 (BFS)

如果这个基本解还满足 $x_B \geq 0$ ，就是 BFS。

4. 约化成本 (reduced cost)

它告诉你：让某个非基变量从 0 增加，会不会让目标值下降。

5. 换基 (pivot)

一个变量入基，一个变量离基，对应从一个 BFS 移动到另一个 BFS。

二、整章最重要的逻辑链条

这一章的逻辑，其实可以压缩成下面这一条链：

当前基 B ; $\implies$; $x_B = B^{-1}b$; $\implies$; $\bar{c}_j = c_j - c_B^T B^{-1} A_j$; $\implies$; 选入基变量

然后继续：

$u = B^{-1} A_j$; $\implies$; $\theta^* = \min_{i: u_i > 0} \frac{x_{B(i)}}{u_i}$; $\implies$; 选离基变量; $\implies$; 得到新基、新 BFS

最后不断重复，直到：

- 所有约化成本都非负 $\implies$ 最优；
- 或出现 $u \leq 0 \implies$ 无界；
- 或在退化情况下需要反循环规则。

这就是第三章最核心的“骨架”。

三、3.1 最优性条件：怎么判断当前 BFS 已经最优

1. 基本思想：看附近有没有下降方向

一般优化里，局部最优不一定是全局最优。

但线性规划很特殊，因为：

- 可行域是凸集；
- 目标函数 $c^T x$ 是线性的。

所以在线性规划中：

只要当前点附近不存在可行下降方向，它就是全局最优。

这就是 3.1 的理论起点。

2. 可行方向 (feasible direction)

如果在可行点 x 处，存在某个 $\theta > 0$ 使得

$$x + \theta d \in P,$$

那么 d 称为 x 处的一个可行方向。

直观上就是：你从当前点往 d 的方向走一小步，仍然留在可行域里。

3. 基本方向 (basic direction)

如果当前点是 BFS，那么所有非基变量都为 0。
此时最自然的移动方式是：

- 选一个非基变量 x_j ；
- 让它从 0 开始增加；
- 其他非基变量保持 0；
- 基变量自动调整以保持 $Ax = b$ 。

令方向向量满足：

- $d_j = 1$ ；
- 其余非基变量分量为 0；
- 基变量部分由约束决定。

因为要保持 $Ad = 0$ ，所以有

$$Bd_B + A_j = 0,$$

从而

$$d_B = -B^{-1}A_j.$$

这个方向就是第 j 个基本方向。

4. 约化成本 (reduced cost)

沿该方向走，目标函数变化率为

$$c^T d = c_B^T d_B + c_j.$$

代入 $d_B = -B^{-1}A_j$ ，得到

$$c^T d = c_j - c_B^T B^{-1} A_j.$$

定义

$$\bar{c}_j = c_j - c_B^T B^{-1} A_j,$$

这就是变量 x_j 的约化成本。

它的意义非常重要：

- $\bar{c}_j < 0$ ：让 x_j 增加会让目标值下降；
- $\bar{c}_j > 0$ ：让 x_j 增加会让目标值上升；
- $\bar{c}_j = 0$ ：一阶上目标不变。

对最小化问题来说，真正要找的是 **负约化成本**。

5. 为什么基变量的约化成本一定为 0

如果 $x_{B(i)}$ 是基变量，那么 $A_{B(i)}$ 是 B 的第 i 列，因此

$$B^{-1}A_{B(i)} = e_i.$$

于是

$$\bar{c}_{B(i)} = c_{B(i)} - c_B^T B^{-1} A_{B(i)} = c_{B(i)} - c_B^T e_i = 0$$

所以：

真正需要检查的，永远只是非基变量的约化成本。

6. 最优性条件定理

第三章最核心的定理之一是：

- 如果当前 BFS 的所有约化成本都满足 $\bar{c} \geq 0$ ，那么当前 BFS 最优；
- 反过来，如果当前 BFS 最优且非退化，那么必有 $\bar{c} \geq 0$ 。

你要记住它的思想，而不是只记结论：

任意一个可行解，相对于当前 BFS 的偏移，都可以写成非基方向的非负组合；如果每个这样的方向都不能降低目标函数，那任何可行解都不可能更优。

7. 这一节的本质结论

3.1 最终把“最优”变成了一个可检验的代数条件：

$$B^{-1}b \geq 0, \quad \bar{c}^T = c^T - c_B^T B^{-1}A \geq 0.$$

满足这两个条件的基，叫做 **最优基 (optimal basis)**。

四、3.2 单纯形法的推导：如果不最优，怎么走到下一个 BFS

3.1 解决了“停不停”的问题，3.2 解决“怎么走”的问题。

1. 入基变量 (entering variable)

如果当前存在某个非基变量 x_j 使得

$$\bar{c}_j < 0,$$

那么它对应一个可能的下降方向。

于是选它入基。

这一步的含义是：

| 让当前为 0 的变量 x_j 开始变正。

2. 方向向量与主列

令

$$u = B^{-1}A_j.$$

那么

$$d_B = -u.$$

所以如果沿该方向移动 θ ，新的基变量满足

$$y_B = x_B - \theta u.$$

这说明：

- 若 $u_i > 0$ ，则对应基变量会减少；
- 若 $u_i \leq 0$ ，则对应基变量不会先变坏。

3. 最小比值检验 (minimum ratio test)

为了保持非负性，必须有

$$x_{B(i)} - \theta u_i \geq 0.$$

因此对所有 $u_i > 0$ ，都有

$$\theta \leq \frac{x_{B(i)}}{u_i}.$$

所以最大可行步长是

$$\theta^* = \min_{i: u_i > 0} \frac{x_{B(i)}}{u_i}.$$

这就是最小比值检验。

它的本质是：

谁最先降到 0，谁就必须离开基。

4. 离基变量 (leaving variable)

设最小值在第 ℓ 行达到，那么第 ℓ 个基变量离基，入基变量是 x_j 。

于是：

- x_j 入基；
- $x_{B(\ell)}$ 离基；
- 得到新基 $\bar{B}$ ；
- 得到新的 BFS。

5. 无界性的判定

如果对某个负约化成本变量 x_j ，其对应

$$u = B^{-1}A_j \leq 0,$$

那么沿该方向，所有基变量都不会因为增大步长而违反非负约束。

于是可以一直走下去，且因为 $\bar{c}_j < 0$ ，目标值会不断下降。

因此问题无界：

$$\text{optimal cost} = -\infty.$$

6. 非退化情形下为什么有限终止

如果每个 BFS 都非退化，那么每一步都有

$$\theta^* > 0.$$

再加上入基变量满足 $\bar{c}_j < 0$ ，所以每步目标值都严格下降：

$$c^T y = c^T x + \theta^* \bar{c}_j < c^T x.$$

由于 BFS 的个数有限，不可能无限下降又无限换基，因此算法一定有限步终止。

7. 3.2 的一句话总结

3.2 把单纯形法的“一步”完全说清楚了：

找负约化成本变量入基，用 $u = B^{-1}A_j$ 算方向，用比值检验算最大步长，用达到最小比值的基变量离基。

五、3.3 单纯形法的实现：算法逻辑一样，但“记账方式”不同

数学上，simplex 的核心不变；但实现上，差别很大。

1. 朴素实现 (naive implementation)

每一轮都重新解两个线性方程组：

$$p^T B = c_B^T, \quad Bu = A_j,$$

分别得到

$$p^T = c_B^T B^{-1}, \quad u = B^{-1}A_j.$$

然后再算各个 reduced costs。

优点是概念直接；缺点是每轮都“从头算”，成本较高。

2. 修正单纯形法 (revised simplex method)

核心思想是：

不要每轮都重新解方程，而是直接维护当前的 B^{-1} 。

于是：

- 单纯形乘子

$$p^T = c_B^T B^{-1}$$

只需矩阵-向量乘法；

- 主列

$$u = B^{-1}A_j$$

也只需矩阵-向量乘法。

换基后，由于新基只是旧基替换了一列，因此可以通过一次 pivot 对 B^{-1} 做更新，而不是重算整个逆。

这就是 revised simplex 的理论基础。

3. 全表法 (full tableau method)

full tableau 维护的是整个

$$B^{-1}[b \mid A].$$

其中：

- 第 0 列是

$$B^{-1}b = x_B;$$

- 第 j 列是

$$B^{-1}A_j.$$

再加上顶行：

- 左上角为当前目标值的负数；
- 其余为所有变量的约化成本。

于是所有信息都直接写在表里，pivot 时对整张表做高斯消元式更新即可。

4. revised simplex 与 full tableau 的比较

你可以这样记：

full tableau

优点：

- 结构直观；
- 手算、教学、理解 pivot 很方便。

缺点：

- 要存整个 $B^{-1}A$ ；
- 内存开销大；
- 大规模问题不划算。

revised simplex

优点：

- 主要只存 B^{-1} ；
- 更省内存；
- 更适合大规模和稀疏问题；
- 工业实现通常基于它进一步优化。

缺点：

- 观感不如 tableau 直观；
- 手算时不如全表法方便。

5. 这一节真正要理解的点

3.3 的本质不是“背三种方法”，而是理解：

单纯形法的核心计算对象始终是 $B^{-1}A_{j_0}$ 。

因为它同时决定：

- 约化成本；
- 方向；
- 比值检验；
- 换基更新。

六、3.4 退化与反循环：为什么会卡住，怎么保证不停绕圈

这是第三章最容易“理论上没看透”的部分。

1. 退化 (degeneracy) 是什么

退化指的是：某个 BFS 中，有些基变量虽然在基里，但它们的值却等于 0。

也就是：

“基变量”是代数概念，不等于“正变量”。

2. 退化带来的问题

退化会导致：

(1) 基本方向不一定可行

如果某个基变量本来就是 0，而沿方向还要减少，那么任意正步长都会不可行。

(2) 可能出现 $\theta^* = 0$

这意味着做了换基，但点根本没动，目标值也没变。

(3) 可能出现循环 (cycling)

若换基规则不当，就可能若干步后回到原来的 basis，然后无限重复。

3. 为什么“点没动但基变了”仍然有意义

因为 basis 变了以后：

- 非基变量集合变了；
- 下一轮的基本方向变了；
- 后续可能找到新的下降路径。

所以退化 pivot 虽然当前没降成本，但不一定是无意义的。

4. 字典序规则 (lexicographic pivoting rule)

这个规则的思想是：

- 入基列照样选负 reduced cost；
- 离基时，不只是做普通最小比值检验；
- 如果多个行打平，就比较“标准化后整行”的字典序，选字典序最小的一行离基。

它的核心作用是构造一个严格单调量：

每次 pivot 后，第 0 行都按字典序严格增加。

因此不可能回到旧 tableau，也就不可能循环。

5. Bland 规则 (Bland's rule)

这是更简单也更经典的反循环规则：

- 在所有可入基变量中，选下标最小的；
- 在所有可能离基变量中，选下标最小的。

它的证明没有字典序那么“几何化”，但结论同样重要：

Bland 规则可以保证单纯形法不会循环。

6. 这一节你最该记住什么

3.4 的核心认识不是规则细节，而是：

退化使“换基次数”与“实际移动次数”不再一致。

所以非退化时那套“严格降目标值就一定终止”的证明失效了，必须另加反循环机制。

七、3.5 寻找初始 BFS：单纯形法从哪里开始

单纯形法不是无中生有的，它必须从一个 BFS 开始。

1. 容易情况

如果问题来自

$$Ax \leq b, \quad b \geq 0,$$

加入松弛变量后：

$$Ax + s = b, \quad s \geq 0,$$

令 $x = 0, s = b$ ，就直接得到一个 BFS。

2. 一般情况：辅助问题（Phase I）

若原问题的初始 BFS 不明显，就引入人工变量 y ，构造辅助问题：

$$\min \sum_{i=1}^m y_i$$

subject to

$$Ax + y = b, \quad x \geq 0, y \geq 0.$$

此时令

$$x = 0, \quad y = b$$

就是一个显然可行的初始 BFS。

3. 辅助问题为什么能判断原问题是否可行

如果原问题可行，那么存在某个 $x \geq 0$ 满足 $Ax = b$ 。

于是令 $y = 0$ ，辅助问题就能达到目标值 0。

所以：

- 辅助问题最优值 > 0 ：原问题不可行；
 - 辅助问题最优值 $= 0$ ：原问题可行。
-

4. 为什么还要把人工变量赶出基

即使辅助问题最优值为 0，也可能出现这种情况：

- 某些人工变量还留在基里；
- 但它们的值已经是 0。

这说明当前是退化 BFS。

为了进入原问题的 Phase II，需要把这些人工变量逐个换出去，只保留原变量对应的列构成 basis。

如果某行根本找不到可替换列，那通常说明对应约束是冗余约束，可以删掉。

5. 两阶段法（two-phase method）

这是最标准的框架：

Phase I

解辅助问题，目标是：

- 判断原问题是否可行；
- 找到原问题的初始 BFS；
- 驱逐人工变量；
- 清理冗余约束。

Phase II

在 Phase I 得到的 basis 上，换回原目标函数，继续跑普通 simplex。

6. Big-M 法

另一种思路是直接在原目标函数里给人工变量加巨大惩罚：

$$\min c^T x + M \sum_i y_i.$$

这等于把“先清除人工变量，再优化原目标”揉成一个问题。概念上直观，但数值上通常不如 two-phase 干净。

八、3.6 列几何 (column geometry)：从几何上看单纯形法到底在干什么

这一节很美，也很容易在第一次读时觉得抽象。

其实它是在把“basis / pivot / reduced cost”重新解释成几何图景。

1. 基本思想

把每一列 A_i 和对应成本 c_i 拼成一个更高维点：

$$[A_i \ c_i].$$

在线性规划的某个特殊等价形式下，问题变成：

在这些列点的凸包里，找与某条“要求线 (requirement line)”的最低交点。

2. basic simplex

一个 basis 对应若干个 basic points。

这些点的凸包形成一个 **单纯形 (simplex)**，叫做 **basic simplex**。

当前 BFS 对应的，就是 requirement line 与当前 basic simplex 的交点。

3. 换基的几何意义

代数上：一个变量入基、一个变量离基。

几何上：当前 basic simplex 替换掉一个顶点，变成另一个 simplex。

然后 requirement line 与新 simplex 相交于一个新的点。

如果这个交点更低，目标值就下降。

4. reduced cost 的几何意义

当前 basic points 确定一个超平面，教材把它看成某种 **dual plane**。

若某个非基点位于这个平面之下，那么让它入基会降低交点高度，也就是降低目标值。

这个“低于平面的程度”，本质上就对应 reduced cost。

所以你可以记成：

- $\bar{c}_j < 0$ ：该点在当前 dual plane 下方，值得入基；
 - $\bar{c}_j = 0$ ：在平面上；
 - $\bar{c}_j > 0$ ：在平面上方。
-

5. 为什么叫 simplex method，为什么叫 pivot

- **simplex**：因为每个 basis 对应一个单纯形；
- **pivot**：因为你可以把换基想成当前 simplex 围绕某些固定顶点“转动”，替换掉一个顶点。

所以这两个术语都不是偶然命名，而是有几何来源的。

九、3.7 计算效率：为什么它最坏很差，但实际常常很快

这一节是“理论与实践反差”最强的一节。

1. 看效率要看两件事

一个算法快不快，要看：

- 每次迭代的成本；
- 一共需要多少次迭代。

3.3 主要讲前者，3.7 主要讲后者。

2. 实践中通常很快

经验上，simplex method 在大量实际问题中表现非常优秀，常常只需要不多的 pivot 就到最优。

这也是它长期统治工业 LP 求解的重要原因。

3. 但最坏情况可能指数级

教材给出经典坏例子：

存在某些多面体和某些 pivot rule，使得 simplex method 会沿着一条非常长的顶点路径前进，访问指数多个顶点。

因此它的最坏情况复杂度可能是指数级。

这说明：

simplex 不是一个“最坏情况多项式时间算法”。

4. 这不代表它“差”

因为最坏情况只是说明：

- 你可以被某些精心设计的实例坑到很惨；

并不等于：

- 现实问题通常也这么惨。

事实上，现实中它经常远比最坏情况好得多。

5. 多面体直径与 Hirsch 猜想

这一节还把问题引向更深的组合几何：

如果 simplex 只能沿着多面体的边在顶点间走，那么任意两个顶点之间最短要走多少步？

这就引出 **多面体直径 (diameter of polyhedra)** 的问题，以及著名的 **Hirsch conjecture**。

这里你不必死记证明细节，但要知道：

simplex 的复杂度问题，不只是算法问题，也和多面体图结构本身有关。

6. 本节最重要的认识

3.7 最想传达的是：

simplex 的理论最坏性能很差，但实践性能通常很好；两者之间的差距，是线性规划理论里非常经典的主题。

十、整章知识地图：每一节各自解决什么问题

你可以这样记整章：

3.1 最优性条件

解决“什么时候停”

核心：约化成本 $\bar{c}_j$ 。

3.2 单纯形法推导

解决“怎么走到下一个 BFS”

核心：入基、方向、比值检验、离基。

3.3 实现

解决“计算机怎么高效做这些运算”

核心：naive / revised simplex / tableau。

3.4 反循环

解决“退化时为什么会卡住，如何防止循环”

核心：lexicographic rule、Bland rule。

3.5 初始 BFS

解决“算法从哪里开始”

核心：Phase I / Phase II、人工变量、Big-M。

3.6 列几何

解决“这些代数步骤几何上意味着什么”

核心：basic simplex、dual plane、pivot 的几何解释。

3.7 计算效率

解决“它理论上快不快，实践上为什么常常更快”

核心：最坏情况 vs 实际表现。

十一、本章最核心的公式清单

下面这些公式，基本就是第三章最重要的“必背骨架”。

1. 基变量值

$$x_B = B^{-1}b$$

2. 基本方向

若非基变量 x_j 入基，则

$$d_j = 1, \quad d_B = -B^{-1}A_j$$

3. 单纯形乘子

$$p^T = c_B^T B^{-1}$$

4. 约化成本

$$\bar{c}_j = c_j - c_B^T B^{-1}A_j = c_j - p^T A_j$$

5. 成本变化率

$$c^T d = \bar{c}_j$$

6. 主列

$$u = B^{-1}A_j$$

7. 最小比值检验

$$\theta^* = \min_{i: u_i > 0} \frac{x_{B(i)}}{u_i}$$

8. 最优性条件

若所有非基变量满足

$$\bar{c}_j \geq 0,$$

则当前 BFS 最优。

9. 无界性判定

若存在某个 $\bar{c}_j < 0$ 且

$$u = B^{-1}A_j \leq 0,$$

则问题无界。

十二、本章最容易混淆的点

1. “基变量”不等于“正变量”

退化时，基变量也可能等于 0。

2. “约化成本”不等于“原目标系数”

真正决定是否值得入基的是

$$\bar{c}_j,$$

不是 c_j 。

3. 基本方向不一定可行

在非退化 BFS 处通常可行；在退化处未必。

4. 比值检验只看 $u_i > 0$

因为只有这些变量会随着步长增加而减小到 0。

5. 换基次数不一定等于几何上真的走了一条新边

退化时可能“换了 basis，但点没动”。

6. “最优 BFS”和“最优基”在退化时要分开想

一个最优点可能对应多个 basis，其中并不是每个基都一定满足你最喜欢的代数性质。

十三、如果把整章压缩成一句真正的“考试版总结”

可以写成下面这句：

单纯形法利用“线性规划若有最优解则存在最优基本可行解”这一事实，从一个 BFS 出发，通过检查非基变量的约化成本来判断最优性；若存在负约化成本，则令相应变量入基，并由 $u = B^{-1}A_j$ 与最小比值检验确定离基变量，从而换到新的 BFS；在实现上可采用 full

tableau 或 revised simplex，在退化情形下需借助反循环规则，而若初始 BFS 不明显则需通过两阶段法或 Big-M 法构造起点。

附录：

revised simplex method 与 full tableau 的全流程对应

一、先给结论：两种方法到底是什么关系

1. 它们解的是同一个问题

都在解标准形线性规划：

$$\min c^\top x$$

subject to

$$Ax = b, \quad x \geq 0$$

两种方法都围绕同一个当前基矩阵 B 展开，也都依赖下面这些核心量：

当前基变量值：

$$x_B = B^{-1}b$$

单纯形乘子：

$$p^\top = c_B^\top B^{-1}$$

约化成本：

$$\bar{c}_j = c_j - c_B^\top B^{-1}A_j = c_j - p^\top A_j$$

主列向量：

$$u = B^{-1}A_j$$

最小比值检验：

$$\theta^* = \min_{i: u_i > 0} \frac{x_{B(i)}}{u_i}$$

所以从数学本质上看，它们不是两套不同的算法，而是：

同一个单纯形迭代过程的两种不同实现方式。

2. 它们唯一真正的区别：保存信息的方式不同

full tableau

它把当前基下的大量信息都显式摊开保存。

在 full tableau 中，你直接保存：

- 当前基变量值： $B^{-1}b$
- 各列的当前表示： $B^{-1}A_j$
- 各变量的约化成本： $\bar{c}_j$
- 当前目标值： $c_B^\top B^{-1}b$

更具体地说：

- 第 0 列是 $B^{-1}b$
- 第 j 列是 $B^{-1}A_j$
- 顶行第 j 个元素是 $\bar{c}_j$
- 左上角对应当前目标值的负数

所以它的思想是：

把当前基下几乎所有会用到的信息都提前算好并摊在表里。

revised simplex method

它不显式保存整个 $B^{-1}A$ ，而是主要保存：

- 当前 basis
- 当前 B^{-1}
- 当前 $x_B = B^{-1}b$
- 当前 c_B

至于某一列的当前表示，不提前全部存，而是在需要某个变量 x_j 时再算：

$$u = B^{-1}A_j$$

所以它的思想是：

只保留核心引擎 B^{-1} ，其余量按需生成。

二、从原问题开始到最终结果：两种方法的完整流程对应

Step 0：先把原问题化为标准形

无论用 revised simplex method 还是 full tableau，第一步都完全一样：

把原问题化成

$$Ax = b, \quad x \geq 0$$

常见操作包括：

- 加松弛变量
- 自由变量拆成两个非负变量之差
- 必要时把某些约束乘以 -1

这一步两者没有区别，因为此时还没有涉及“当前基如何存储”。

Step 1：找到初始基本可行解

这是单纯形法的起点，两种方法共同需要。

情况 A：初始 BFS 很明显

如果问题整理后是

$$Ax + s = b, \quad b \geq 0, \quad s \geq 0$$

那么直接取

$$x = 0, \quad s = b$$

就得到一个初始 BFS。

这时初始基通常就是松弛变量对应的单位阵。

情况 B：初始 BFS 不明显

这时需要做 Phase I，构造辅助问题：

$$\min \sum_{i=1}^m y_i$$

subject to

$$Ax + y = b, \quad x \geq 0, \quad y \geq 0$$

先让人工变量 y 构成一个显然可行的初始基，跑完辅助问题后，再进入原问题的 Phase II。

这一步的数学逻辑，对两种方法完全相同。

这一阶段两者的差异

full tableau

直接给辅助问题写出初始 tableau。

如果人工变量对应单位阵，那么初始表会非常整齐：

- 第 0 列直接是初始基变量值
- 其他列直接写入
- 顶行直接反映辅助目标函数

revised simplex method

不把整张表写出来，而是记录：

- 当前基变量下标
- 当前 B^{-1}
- 当前 $x_B = B^{-1}b$
- 当前 c_B

所以这一阶段可以这样理解：

- full tableau：把状态摊开
- revised simplex：把状态压缩

Step 2：一轮迭代开始前，当前状态分别是什么

这一点最关键。

full tableau 当前手里有什么

你可以直接读到：

- 当前基变量值：第 0 列
- 当前第 j 列： $B^{-1}A_j$
- 当前约化成本：顶行
- 当前目标值：左上角

所以 full tableau 的特点是：

当前基下的大量信息已经显式写出来了。

revised simplex method 当前手里有什么

它通常只保存：

- 当前基变量集合
- 当前 B^{-1}
- 当前 $x_B = B^{-1}b$
- 当前 c_B

如果要知道某个变量 x_j 的列表示，就现算：

$$u = B^{-1}A_j$$

如果要知道某个变量的约化成本，就现算：

$$\bar{c}_j = c_j - c_B^\top B^{-1}A_j$$

所以 revised simplex 的特点是：

不把所有量都摊开，而是只保留产生这些量的核心对象。

Step 3：检查最优性

这一步两者做的是同一件事：

看是否存在某个非基变量满足

$$\bar{c}_j < 0$$

如果不存在这样的变量，那么当前解最优。

full tableau 怎么做

直接看顶行。

因为顶行第 j 个元素就是 $\bar{c}_j$ ，所以：

- 若顶行中所有非基变量对应元素都非负，则最优
 - 若有负元素，则继续迭代
-

revised simplex method 怎么做

先算单纯形乘子：

$$p^\top = c_B^\top B^{-1}$$

再对候选非基变量逐个计算：

$$\bar{c}_j = c_j - p^\top A_j$$

然后检查是否存在负约化成本。

这一阶段的具体对应

- full tableau：直接读顶行
- revised simplex：先算 $p^\top$ ，再算 $\bar{c}_j$

但它们比较的是同一个量：约化成本。

Step 4：选择 entering variable

若存在 $\bar{c}_j < 0$ ，就选一个变量入基。

这一点两种方法完全相同。

都可以使用各种选主元规则，例如：

- first negative reduced cost
- most negative reduced cost
- Bland rule

区别只在于：

- full tableau：负约化成本在表里直接能看到
 - revised simplex：负约化成本需要先算出来
-

Step 5：形成主列向量

这一步是两者最关键的对应之一。

选定入基变量 x_j 后，都需要研究：

$$u = B^{-1}A_j$$

revised simplex method 里

它是显式计算出来的：

$$u = B^{-1}A_j$$

然后：

- 它给出基变量方向变化：

$$d_B = -u$$

- 它决定比值检验
 - 它决定谁离基
-

full tableau 里

你根本不用再算。

因为在 tableau 里，变量 x_j 对应的那一列本来就是：

$$B^{-1}A_j$$

也就是说，主列本身就是 u 。

这一阶段最重要的对应

revised simplex 中现算的 $u = B^{-1}A_j$ ，
就是 full tableau 中直接读出来的主列。

Step 6：无界性检测

选定入基变量后，检查主列 u 的符号。

如果

$$u_i \leq 0, \quad \forall i$$

那么沿该方向前进时，所有基变量都不会先碰到 0，而目标值又在下降，所以问题无界。

revised simplex method 怎么做

直接检查现算出来的 u 是否有正分量。

- 若没有正分量，则无界
-

full tableau 怎么做

直接检查主列是否有正元素。

- 若没有正元素，则无界
-

这一阶段的对应

revised simplex 检查的是向量 u 的正分量，
full tableau 检查的是主列中的正元素。

它们本质上是同一件事。

Step 7: 最小比值检验，确定 leaving variable

若主列 u 中有正分量，则做 ratio test。

对所有满足 $u_i > 0$ 的行，计算：

$$\frac{x_{B(i)}}{u_i}$$

取最小者，对应的行就是离基行。

revised simplex method 怎么做

它已经有：

$$x_B = B^{-1}b$$

以及

$$u = B^{-1}A_j$$

于是直接计算：

$$\frac{x_{B(i)}}{u_i}$$

即可。

full tableau 怎么做

在 tableau 里：

- 第 0 列就是 $x_B = B^{-1}b$
- 主列就是 $u = B^{-1}A_j$

所以 ratio test 就是：

第 i 行的比值等于

$$\frac{\text{第 0 列第 } i \text{ 行}}{\text{主列第 } i \text{ 行}}$$

这和

$$\frac{x_{B(i)}}{u_i}$$

完全一致。

这一阶段的对应

revised simplex 用 x_B 和 u 算比值，
full tableau 用“第 0 列 ÷ 主列”算比值。

它们逐项一一对应。

Step 8: 更新新的 BFS

设离基行为 ℓ ，步长为

$$\theta^* = \frac{x_{B(\ell)}}{u_\ell}$$

那么新解满足：

入基变量变为

$$y_j = \theta^*$$

保留下来的基变量变为

$$y_{B(i)} = x_{B(i)} - \theta^* u_i, \quad i \neq \ell$$

离基变量变为 0。

revised simplex method 怎么做

它通常显式地用上面的公式更新当前基变量值。

full tableau 怎么做

它不单独写这个公式，而是通过一次 pivot 行变换自动完成。

但本质上，pivot 后新表的第 0 列，恰好就是新的

$$\bar{B}^{-1}b$$

也就是新基变量值。

这一阶段的对应

revised simplex 是“按公式显式更新新 x_B ”，
full tableau 是“通过 pivot 后让新 x_B 自动出现在第 0 列”。

Step 9: 更新 basis 本身

这一步两种方法完全一样。

若入基变量是 x_j ，离基变量是 $x_{B(\ell)}$ ，那么新的基就是把旧基中的第 ℓ 列换成 A_j 。

新基可写为：

\$\$

$\bar{B}$

$[A_{B(1)}, \dots, A_{B(\ell-1)}, A_j, A_{B(\ell+1)}, \dots, A_{B(m)}]$

这一步没有实现差异，它是纯粹的算法逻辑。 --- # Step 10: 更新“当前基下的全部信息”这是两

$$u = B^{-1} A_j$$

做一次主元更新。记某个更新矩阵为 Q ，则有：

$$\bar{B}^{-1} = Q B^{-1}$$

更新完成后，它得到新的： - basis - $\bar{B}^{-1}$ - 新的 x_B 但它仍然不会显式形成整块 $\bar{B}^{-1}b$

$$p^{\text{top}} = c_B^{\text{top}} B^{-1}$$

再算

$$\bar{c}_j = c_j - p^{\text{top}} A_j$$

当前目标值则是

$$c_B^{\text{top}} x_B = c_B^{\text{top}} B^{-1} b$$

所以它的顺序是：1. 先更新 B^{-1} 2. 再由 B^{-1} 推出顶层信息 --- ## full tableau 怎么做这

$$\bar{c}_j \geq 0$$

则当前 BFS 最优。 --- ## 情况 2：无界 如果选定某个负约化成本变量后，对应主列没有正分量，则

$$x_B = B^{-1} b$$

- revised simplex: 维护或现算 - full tableau: 第 0 列直接可见 --- ## 对应 2: 约化成本

$$\bar{c}_j = c_j - c_B^{\text{top}} B^{-1} A_j$$

- revised simplex: 由 $p^{\text{top}} = c_B^{\text{top}} B^{-1}$ 进一步计算 - full tableau: 顶行直接给出 ---

$$u = B^{-1} A_j$$

- revised simplex: 现算 - full tableau: 主列本身就是它 --- ## 对应 4: 比值检验

$$\theta^* = \min_{i: u_i > 0} \frac{x_{B(i)}}{u_i}$$

- revised simplex: 用 x_B 与 u 算 - full tableau: 用“第 0 列 ÷ 主列”算 --- ## 对应 5: 1

$$y_{B(i)} = x_{B(i)} - \theta^* u_i$$

- revised simplex: 显式按公式更新 - full tableau: pivot 后第 0 列自动更新 --- ## 对应 6: 更新

$$p^{\text{top}} = c_B^{\text{top}} B^{-1}$$

以及

$$\bar{c}_j = c_j - p^{\text{top}} A_j$$

然后进入普通单纯形迭代。 --- # 六、你可以怎样一句话理解两者 ## full tableau > 每一轮都把当